

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUA – PEC 1

Presentación

Esta primera PEC sirve como introducción a las arquitecturas de software para el desarrollo de aplicaciones distribuidas donde se trabajan los cimientos, técnicas y habilidades aplicables en la definición moderna de arquitecturas del software y se profundiza en la selección del estilo arquitectónico más adecuado según la tipología de la aplicación a desarrollar.

Para elaborar la actividad se formulan cuatro ejercicios: el primer ejercicio evaluará alguno de los aspectos fundamentales en la definición de una arquitectura, mientras que el segundo y el tercer ejercicio habrá que decidir y razonar el estilo arquitectónico (o mezcla de estilos) más adecuado en función de los requisitos planteados en un caso de estudio. Finalmente, el último ejercicio evaluará algunos conceptos básicos de las arquitecturas distribuidas.

La PEC 1 alcanza los contenidos del libro *Fundamentals of Software Architecture* (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PEC se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

Los objetivos de la PEC 1 son:

- Explicar los fundamentos de la arquitectura de software y las características de una arquitectura.
- Exponer argumentos sobre los puntos fuertes y los puntos débiles de los diferentes estilos arquitectónicos y como estos influyen en la decisión del estilo o estilos arquitectónicos más adecuados para una determinada aplicación.

- Situar los nuevos paradigmas emergentes de la ingeniería del software dentro del marco de las arquitecturas del software.

Descripción de la PEC a realizar

Ejercicio 1 (2 puntos)

Realizad un estudio comparativo de los estilos arquitectónicos *Service Based* y *Event Driven* en función de las características arquitectónicas de cada uno de ellos. Usad la tabla a continuación.

Por último, proponed un ejemplo de aplicación que encaje en cada uno de los dos estilos arquitectónicos.

	<i>Service Based</i>	<i>Event Driven</i>
<i>Partitioning type</i>		
<i>Number of quanta</i>		
<i>Deployability</i>		
<i>Elasticity</i>		
<i>Evolutionary</i>		
<i>Fault tolerance</i>		
<i>Modularity</i>		
<i>Overall coste</i>		
<i>Performance</i>		
<i>Reliability</i>		
<i>Scalability</i>		
<i>Simplicity</i>		
<i>Testability</i>		

Solución

	<i>Service Based</i>	<i>Event Driven</i>
<i>Partitioning type</i>	Separación por elementos del dominio.	Separación puramente tecnológica.
<i>Number of quanta</i>	Dependiendo de la implementación pueden variar de uno a diversos.	Dependiendo de la implementación pueden variar de uno a diversos.

<i>Deployability</i>	Habilidad de desplegar versiones frecuentemente. El hecho de tener las funcionalidades particionadas por elementos del dominio hace que los despliegues sean algo más fáciles.	Habilidad de desplegar versiones frecuentemente.
<i>Elasticity</i>	Muy poca elasticidad debido a que los servicios son <i>coarse-grained</i> .	Algo mejor en este aspecto, pero tampoco es su fuerte.
<i>Evolutionary</i>	Es algo más complicado añadir nuevas funcionalidades.	Excelente para añadir nuevas funcionalidades, ya sea a procesadores de eventos existentes o bien creando de nuevos.
<i>Fault tolerance</i>	Alta tolerancia a fallos puesto que normalmente los diferentes servicios suelen estar autocontenidos y la caída de uno de ellos no afecta al resto.	Tolerancia a fallos máxima que se consigue intermediando <i>event processors</i> altamente desacoplados y asíncronos que proporcionan consistencia y procesamiento eventual de los <i>event workflows</i> .
<i>Modularity</i>	Buena modularidad aunque no tanto como una arquitectura con servicios más <i>fine-grained</i> .	Buena modularidad.
<i>Overall coste</i>	Coste moderado.	Coste mucho más elevado.
<i>Performance</i>	Puede ofrecer un rendimiento aceptable pero no llega al que se puede conseguir con <i>Event Driven</i> .	Rendimiento máximo conseguido con comunicaciones asíncronas combinadas con procesamiento paralelo.
<i>Reliability</i>	Buena <i>reliability</i> por el hecho que los servicios son más <i>coarse-grained</i> y tienen que hacer menos uso de la red.	Algo menos <i>reliable</i> .
<i>Scalability</i>	Moderada principalmente porque los servicios su <i>coarse-grained</i> .	Escalabilidad más elevada mediante técnicas como los balanceos programáticos de los procesadores de eventos
<i>Simplicity</i>	A pesar de no ser trivial, es una arquitectura mucho más simple	Muy compleja. La complejidad es uno de los <i>drawbacks</i> principales.
<i>Testability</i>	Altamente testeable por el hecho de tener servicios autocontenidos.	Relativamente poco debido a la naturaleza no determinista y dinámica de los flujos de eventos

Un ejemplo de aplicación que encaje en un estilo arquitectónico *Service Based* puede ser una aplicación móvil de gestión de alquileres que acceda a los diferentes servicios mediante una API REST. Cada uno de los servicios es independiente y trabaja en su propio dominio (usuarios, alquileres, facturas, etc.).

Un ejemplo de aplicación que encaje en un estilo arquitectónico *Event Driven* podría ser un juego en que los personajes reaccionen a diferentes eventos.

Ejercicio 2 (3 puntos)

Durante la época de pandemia muchos negocios se han visto obligados a reinventarse para no tener que desaparecer. Uno de ellos ha sido el mundo de la restauración. Un grupo de restauración que cuenta con 3 restaurantes en la ciudad de Girona ha decidido potenciar el reparto a domicilio.

Para hacerlo, aparte de la atención telefónica, han decidido crear una aplicación web muy sencilla que tenga los platos diarios y permite a los clientes hacer el pedido. Para implementar la aplicación se ha contratado un pequeño equipo de desarrollo propio con 2 expertos en UI, 5 desarrolladores Java y 1 experto en Oracle. El miembro más experimentado del equipo de desarrollo hará también de arquitecto.

El despliegue de toda la aplicación se hará en un servidor de aplicaciones JBoss y los datos se persistirán en una base de datos relacional Oracle. Tanto el servidor de aplicaciones como la base de datos están instalados en una infraestructura "on-premise".

Por último, hay que tener en cuenta que la situación económica del grupo de restauración es bastante crítica, por eso nos interesa mucho más tener algo funcionando rápidamente y con un coste bajo, que pensar en posibles problemas de rendimiento, mantenibilidad o modularidad.

¿Qué estilo arquitectónico de los que habéis estudiado en el Capítulo II: *Architecture Styles* del libro *Fundamentals of Software Architecture* creéis que encaja más con la naturaleza del proyecto? Razonad la respuesta comparando los requerimientos y restricciones del proyecto a desarrollar con los pros y contras del estilo arquitectónico elegido.

Solución

El estilo arquitectónico que encaja más para desarrollar el proyecto sería un estilo arquitectónico por capas (***Layered Architecture Style***). Algunos de los motivos su los siguientes:

- Según los requerimientos se trata de hacer una aplicación con acceso web muy sencilla, que hay que hacer con una tecnología conocida, con un coste bajo y disponible rápidamente. El

estilo arquitectónico en capas está principalmente enfocado a resolver este tipo de aplicaciones: *"This style of architecture is the de facto standard for most applications, primarily because of its simplicity, familiarity, and low cost"*

- Teniendo en cuenta la composición del equipo de desarrollo, sin expertos en el dominio, la separación por capas sería una separación "tecnológica" y esta separación es precisamente la que tenemos en un estilo arquitectónico en capas: *"The layered architecture is a technically partitioned architecture (as opposed to a domain-partitioned architecture)"*
- La separación en capas encaja perfectamente con una topología basada en una interfaz web, un servidor de aplicaciones para la capa de negocio y un servidor de base de datos separado. Se trata de una topología típica en este estilo arquitectónico.
- El hecho que el despliegue sea *"on-premise"* también encaja en un estilo arquitectónico en capas, es el modelo típico por estas aplicaciones.

Algunos de los puntos débiles de estilo arquitectónico en capas son la modularidad, la escalabilidad, el rendimiento y la mantenibilidad, pero los requisitos nos explicitan que estas características son poco importantes en este estadio del proyecto.

Ved el Capítulo 10. *Layered Architecture Style* del libro *Fundamentals of Software Architecture* para una explicación más extensa de la arquitectura propuesta y como encaja en los requisitos del ejercicio.

Ejercicio 3 (3 puntos)

El grupo inversor que puso en marcha el catálogo de ocio virtual ("**CultureInDaHouse**") que os adjuntamos con este enunciado, consiguió poner en marcha la aplicación en un tiempo récord y esto les ha permitido superar con éxito la crisis. No solo esto, sino que la iniciativa ha tenido muy buena acogida; sus usuarios ya se cuentan por millones y cuentan con más de 1.000 entidades organizadoras en las principales ciudades de Europa. El éxito ha sido tan grande que, incluso, se plantean hacer el salto al mercado americano y asiático.

Pero no todo es tan bonito, debido a este rápido crecimiento ya se están empezando a encontrar algunos problemas y nuevos retos con la aplicación actual de pedidos, los enumeramos brevemente:

- Hay actuaciones en que el número de usuarios que quiere hacer la reserva es muy elevado, en este escenario la aplicación ya va muy lenta y los servidores *"on-premise"* van muy saturados. La elevada concurrencia empieza a ser un problema grave para la plataforma.
- Hay procesos complejos que se resuelven mejor usando técnicas de programación funcional con lenguajes de programación que no son Java. Lo mismo nos pasa con las fuentes de datos, algunos procesos tienen un diseño más natural si usan una base de datos no relacional como, por ejemplo, MongoDB.
- Cada vez hay procesos más complejos y especializados que requieren un dominio más específico tanto a nivel funcional como a nivel técnico. Ya no es posible que un equipo de desarrollo pequeño domine todos estos conceptos.
- Nos ha aparecido el famoso *"Time to market"*, para ser competitivos necesitamos desarrollar nuevas funcionalidades y ponerlas en producción muy rápidamente. Idealmente, una funcionalidad tendría que promocionar a producción de forma segura sin intervención manual,

y este ciclo se podría repetir varias veces al día. Esto último implica que los despliegues de las nuevas funcionalidades tienen que ser independientes y seguir su propio ciclo de vida.

- Nos hemos encontrado demasiadas veces con errores de regresión, y no conseguimos una batería de *tests* automáticos que nos asegure que las *releases* funcionarán correctamente y no romperán nada.

¿Qué estilo arquitectónico de los que habéis estudiado al Capítulo II: *Architecture Styles* del libro *Fundamentals of Software Architecture* creéis que encaja con los nuevos requerimientos del proyecto?

Razonáis la respuesta comparando los nuevos requerimientos y restricciones del proyecto a desarrollar con los pros y contras del estilo arquitectónico que elegís.

Solución

El estilo arquitectónico hacia el que tiene que evolucionar el proyecto es un estilo arquitectónico basado en microservicios (***Microservices Architecture***). Algunos de los motivos son los siguientes:

- Una arquitectura basada en microservicios es altamente escalable y, desplegada en una plataforma adecuada al Cloud, permitirá solventar los problemas de rendimiento con un coste razonable usando técnicas de autoescalado.
- Una arquitectura basada en microservicios permite utilizar la mejor tecnología para cada servicio y que estos colaboren entre ellos.
- Los microservicios pueden tener dominios y funcionalidades altamente especializadas, pero tienen que ser como una caja negra para el resto del sistema. Esto soluciona el problema de especialización mencionado, pasando de una separación puramente "tecnológica" de las capas a una separación por "conceptos de dominio" propia de las arquitecturas basadas en microservicios.
- Un microservicio evoluciona y se puede desplegar de forma independiente del resto de componentes del ecosistema. Esto permite desarrollar nuevas funcionalidades y ponerlas en producción muy rápidamente
- Una de las bases de una arquitectura basada en microservicios es la "testeabilidad", donde cada microservicio tiene su batería de *test* automáticos que asegurarán que no se introduzcan errores de regresión en los despliegues.

Algunos de los puntos débiles de estilo arquitectónico basado en microservicios son la complejidad, el rendimiento por la naturaleza distribuida y el coste en el desarrollo. También hay que mencionar que el uso eficiente de microservicios requiere una cultura DevOps madura y una metodología de desarrollo ágil.

Ved el Capítulo 17: *Microservices Architecture* del libro *Fundamentals of Software Architecture* para una explicación más extensa y detallada de la arquitectura propuesta y su ensambladura con las restricciones y los nuevos requerimientos del proyecto.

Ejercicio 4 (2 puntos)

Responded brevemente a las siguientes preguntas sobre los conceptos vistos en el libro *"Fundamentals of software architecture"*.

4.1 (0.5 puntos) Describe brevemente cada uno de los 4 prismas o dimensiones principales que conforman la definición de una arquitectura, y pone un ejemplo de cada uno de ellos. Los ejemplos elegidos no pueden aparecer en el libro *Fundamentals of Software Architecture*.

Solución

Los 4 prismas o dimensiones principales que conforman la definición de una arquitectura son los siguientes:

- **Características:** *"define the success criteria of a system, which is generally orthogonal to the functionality of the system"*. Un posible ejemplo podría ser el sistema tiene que ser seguro y solo los usuarios autorizados tienen que poder acceder.
- **Decisiones:** *"define the rules for how a system should be constructed"*. Un posible ejemplo podría ser que todas las capas se tienen que comunicar mediante interfaces muy definidas.
- **Estructura:** *"refers to the type of architecture style (or styles) the system is implemented in"*. Un posible ejemplo podría ser que el sistema seguirá un estilo arquitectónico basado en microservicios.
- **Principios de diseño:** *"A design principle differs from an architecture decision in that a design principle is a guideline rather than a hard-and-fast rule"*. Un posible ejemplo podría ser que es preferible comunicar los diferentes servicios mediante una API REST.

Ved el Capítulo 1: *Introduction* del libro *Fundamentals of Software Architecture*.

4.2 (0.5 puntos) Elegid dos expectativas que creáis que están asociadas al rol de arquitecto y poned un ejemplo de tarea concreta por cada una de ellas. Los ejemplos elegidos no pueden aparecer en el libro *Fundamentals of Software Architecture*.

Solución

- **Analizar continuamente la arquitectura:** *Evaluar las alternativas de despliegue al Cloud para ver si hay que hacer una migración en este sentido.*
- **Tener cierto dominio del negocio:** *En el negocio bancario, conocer qué es una hipoteca inversa.*

Ved el Capítulo 1: *Introduction* del libro *Fundamentals of Software Architecture*.

4.3 (0.5 puntos) Cuál de estas tareas asociadas son responsabilidades de un arquitecto y qué son responsabilidad del equipo de desarrollo:

- Realizar un diagrama de clases del sistema de compra de entradas.
- Definir como se gestiona el incremento de tráfico por el *Black Friday*.

- Realizar un diagrama de secuencia del proceso de compra de entradas.
- Realizar un diagrama de componentes con los módulos que tendrá el sistema.
- Elegir los patrones de diseño que seguirá el proceso de compra de entradas.

Solución

- Realizar un diagrama de clases del sistema de compra de entradas: **Equipo de desarrollo.**
- Definir como se gestiona el incremento de tráfico por el *Black Friday*: **Arquitecto.**
- Realizar un diagrama de secuencia del proceso de compra de entradas: **Equipo de desarrollo.**
- Realizar un diagrama de componentes con los módulos que tendrá el sistema: **Arquitecto.**
- Elegir los patrones de diseño que seguirá el proceso de compra de entradas: **Equipo de desarrollo**

Ved el Capítulo 2: *Architectural Thinking - Architecture Versus Design* del libro *Fundamentals of Software Architecture*.

4.4 (0.5 puntos) Explicad porqué creéis que no es posible maximizar TODAS las características deseables para una arquitectura y poned un ejemplo que lo ilustre.

Solución

No es posible maximizar TODAS las características deseables para una arquitectura puesto que muchas veces estas características entran en contradicción y se tiene que llegar a un compromiso en el grado de compleción de cada una de ellas.

Un ejemplo muy típico se la seguridad y el rendimiento, contra más seguridad exigimos menos rendimiento tendrá el sistema.

Ved el Capítulo 4. *Architecture Characteristics Defined* del libro *Fundamentals of Software Architecture*.

Recursos

Recursos Básicos

- Libro "Fundamentals of Software Architecture" (Richards & Ford): Chapter 1. Introduction.
- Libro "Fundamentals of Software Architecture" (Richards & Ford): I Foundations (Chapters 2, 4, 5 y 8)
- Libro "Fundamentals of Software Architecture" (Richards & Ford): II Architecture Styles (Chapters 9 - 14, 16 - 18)

Recursos Complementarios

- Vídeo "Learning Path: Software Architecture Fundamentals—Architectural Thinking"
- Vídeo "Learning Path: Software Architecture Fundamentals—Architecture Styles"
- Libro "Fundamentals of Software Architecture" (Richards & Ford): Appendix-->Self-assessment questions.

Criterios de evaluación

- La PEC se tiene que resolver de forma **estrictamente individual**. En caso de detectar copias se penalizará la actividad con una D como nota.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que enviar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PEC1.pdf*.

Este documento se enviará al espacio de Registro de Evaluación Continuada (REC) del aula antes de las **23:59 horas del día 18 de marzo de 2022**. No se aceptarán entregas fuera de plazo.